MIT 8103 Advanced Database Systems

Ride-Hailing Trip and Payment Records · Emike Isogie · Student ID 301896878

Case study

My case study is Ride-Hailing Trip and Payment Records. I designed the database for dispatchers reviewing trip assignments, analysts summarizing trip activity and payment operators recording trip charges, with separate tables named rider, driver, trip and payment. It supports these operations: create trips and assign available drivers; review trips by status, time range, and rider; summarize trip counts and fares by pickup zone; record a payment for a trip; benchmark trip time-range filtering on 20000 synthetic rows by comparing a table scan with a B-tree index using the same predicate and reporting the query plan and elapsed time.

Portfolio 1: Database Design and Modelling

I organised the relational design into 4 tables. Primary keys identify records, foreign keys preserve the links, and checks enforce the stated business rules. A partial UNIQUE constraint on driver_id applies only when status is assigned or in_progress, preventing a driver from having two active trips. The normalisation examples show the sample data at 1NF, 2NF and 3NF. All 30 deliberately invalid inserts were rejected without changing the valid sample rows.

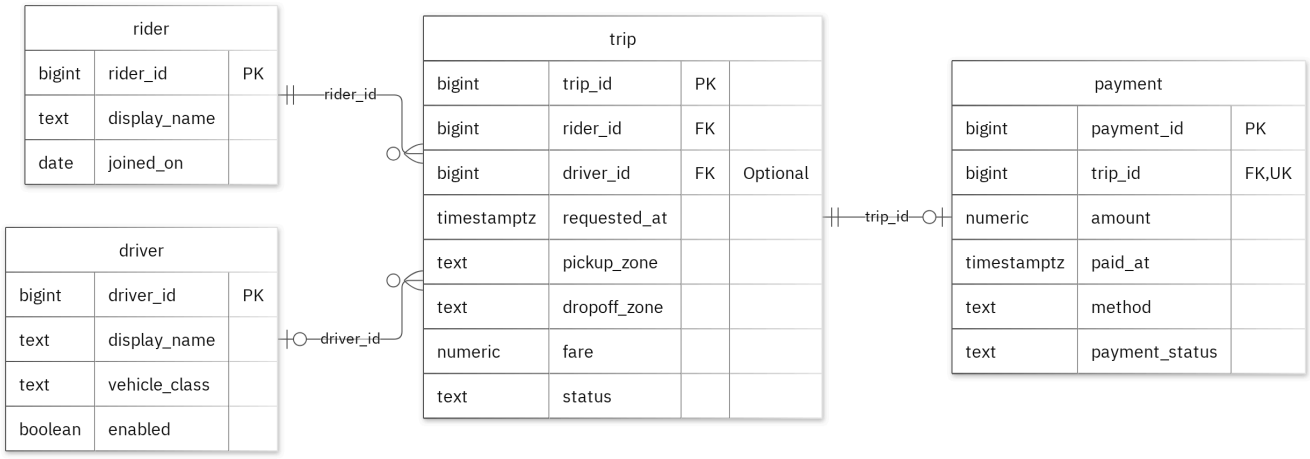


Figure 1. Tables, keys and relationships. PK: primary key; FK: foreign key; UK: unique value.

Portfolio 2: Query Processing and Optimisation

I ran three SQL queries against the sample records. The join returned 1 rows and included Avery Quinn with Morgan Lee. For the index comparison, I used 20,000 separate performance rows. The median lookup time changed from 0.991 ms to 0.178 ms. Both phases returned the same records. The measured plan used the new index. These timings apply to this lookup and environment.

Portfolio 3: Transactions and Concurrency

I used a temporary counter on trip to test COMMIT, ROLLBACK and READ COMMITTED isolation. The uncommitted update remained hidden from the other session, and ROLLBACK restored the earlier committed value. Without locking, two increments left the counter at 2. With SELECT FOR UPDATE, Session B waited for Session A and the final value was 3. The helper is confined to this exercise; the case's original business fields stay unchanged.

Portfolio 4: NoSQL and Advanced Data Models

I used trip_events for the case's event documents. MongoDB inserted 2 sample events and returned the expected results for 4 queries. Validation rejected an invalid date type and a duplicate event ID. Embedded snapshots keep related details with an event, while the details field can differ by event type. These are fictional events; PostgreSQL continues to enforce its relational rules.

Repository and evidence

https://github.com/EmikeIs/Isogie_Emike_301896878

The portfolio folders contain the SQL and JavaScript, explanations, query plans, result tables and screenshots.

Portfolio 5: Distributed and Cloud Database Exercise

I proposed three MongoDB replica members across separate failure zones. The discussion covers primary elections, majority acknowledgement, secondary lag and sharding trade-offs for trip_events. Replication supports availability, while backups address recovery. PostgreSQL needs its own availability plan. This is the permitted discussion option; no live replica set or failover test was deployed.

Individual reflection

I kept rider, driver, trip, payment in separate tables so their details could be maintained without repeating them in every related record. A partial UNIQUE constraint on driver_id applies only when status is assigned or in_progress, preventing a driver from having two active trips. The constraint tests check this alongside the required values and record links. One issue illustrated by the two-session exercise is that both sessions can read the same value before either update is committed. Locking the row before reading preserves both increments. The index comparison also shows why I need to check the execution plan as well as the final answer: the same result can require different amounts of database work.

AI-use declaration

I used GitHub Copilot and the portfolio generator to help prepare the database model, scripts and explanations. The generator ran the PostgreSQL and MongoDB exercises, checked their outputs and captured the result images. I remain responsible for checking and explaining the submitted work.

References

- PostgreSQL: constraints
- PostgreSQL: EXPLAIN
- PostgreSQL: isolation
- PostgreSQL: row locks
- MongoDB: data modelling
- MongoDB: replication
- MongoDB: sharding
- Mermaid: ER diagrams